Identifiers

In Java, the name of a **variable, class, method, or object** is called an **identifier**.

Rules for Identifiers

- An identifier **must not start with a digit**
- It may contain:
  - Letters (`A-Z`, `a-z`)
  - Digits (`0-9`)
  - Underscore (_)
- **Uppercase and lowercase letters are different**
  - `total` and `Total` are **different identifiers**
- The symbol `$` is allowed, but it is **reserved for special purposes** and should generally be avoided
- Cannot use Java keywords

Identifier Rule Examples

Type	Example	Valid
Variable	`count`	✅
Variable	`2count`	❌ (starts with digit)
Class	`Student`	✅
Method	`calculateTotal()`	✅
Variable	`totalScore`	✅
Variable	`total_score`	✅
Variable	`total-score`	❌ (hyphen not allowed)
Variable	`Total` vs `total`	✅ (different identifiers)

Naming Conventions (Common Practice)

Although Java does not strictly enforce these rules, common programming practice follows these conventions:

- **Class names** start with an **uppercase letter**
- **Variables, objects, and method names** start with a **lowercase letter**
- Names usually use **only letters and digits**

Variable

A variable is a storage location that holds data.

Declaration of variable

Syntax

```
Type variable1, variable2, ...;
```

- `Type` specifies the kind of data the variable will store
- Multiple variables of the same type can be declared in one statement
- Variables are separated by commas
- Every declaration ends with a semicolon (`;`)

Example:

```
int count, numberOfStudent, numberOfFaculty;
char answer;
double speed, distance;
```

Type of variable

- **Primitive types:**
  - `int`, `short`, `long` (whole numbers)
  - `float`, `double` (decimals)
  - `char` (single Unicode character)
  - `boolean` (true/false)
  - `byte` (range -128 to 127; often used for file or raw data handling)
- **Non-primitive types:** Strings, Arrays, Objects

Primitive types

Type Name	Kind of Value	Memory Used	Range
`boolean`	true or false	1 byte	Not applicable
`char`	Single character (Unicode)	2 bytes	Common Unicode characters
`byte`	Integer	1 byte	−128 to 127
`short`	Integer	2 bytes	−32,768 to 32,767
`int`	Integer	4 bytes	−2,147,483,648 to 2,147,483,647
`long`	Integer	8 bytes	−9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
`float`	Floating-point number	4 bytes	$1.40239846 \times 10^{-45}$ to $3.40282347 \times 10^{38}$
`double`	Floating-point number	8 bytes	$4.94065645841246544 \times 10^{-324}$ to $1.79769313486231570 \times 10^{308}$

Assignment Statements

An assignment statement evaluates the expression on the right-hand side and stores the result in the variable on the left-hand side.

```
// variable = expression;
count = 0;
count = count + 1;
distance = rate * time;
```

Initialize Variables

Uninitialized variables may sometimes have default values, but this is **not guaranteed** and should not be relied on.

```
int count; // uninitialized variable
int total = 1; //declare + assign initial value
String name = "Bob";
char firstChar = 'B';
```

Default Values in Java

- Default values are automatically assigned to class-level variables if you do not explicitly initialize them.
- Local variables inside methods do NOT get default values — you must assign them a value before using them.
- default values

Type	Default Value
`byte`, `short`, `int`, `long`	`0`
`float`, `double`	`0.0`
`char`	`'\u0000'` (null character)
`boolean`	`false`
Object references (e.g., `String`)	`null`

Arithmetic Operators

- `+` (addition)
- `-` (subtraction)
- `*` (multiplication)
- `/` (division)
- `%` (remainder)

General Rules

- Operands are **promoted before calculation**.

```
byte → short → int → long → float → double
```

- Java evaluates arithmetic expressions using the **widest required type**.

```
byte a = 10;
byte b = 20;

// ❌ Compile-time error
byte c = a + b;   // a + b is promoted to int, not byte

byte a = 5;
short b = 10;
int c = a + b;      // calculation done as int ✔
int x = 5;
double y = 2.5;
double z = x + y; // calculation done as double ✔
```

Floating-point calculations are not exact → use `BigDecimal` when exact precision is required

```
double a = 0.1;
double b = 0.2;

System.out.println(a + b);   // 0.30000000000000004
BigDecimal x = new BigDecimal("0.1");
BigDecimal y = new BigDecimal("0.2");

System.out.println(x.add(y)); // 0.3
```

Parentheses and Precedence Rules

- **Parentheses** are used to explicitly control the order of evaluation in an expression.
- Fully parenthesized expressions remove any ambiguity.

```
int result = 1 + 2 * 3 + 4;
```

unary operator

Operator	Description	Example	Output
`++i`	Pre-increment: increment first, then use value	`int i = 5; System.out.println(++i);`	6
`i++`	Post-increment: use value first, then increment	`int i = 5; System.out.println(i++);`	5
`!x`	Logical NOT	`boolean x = true; System.out.println(!x)`	false

Division operator

- **Integer division** truncates decimals;
- **double division** keeps decimals

Type Casting

- **Widening (automatic):** small → large type (`int → double`)
- **Narrowing (manual):** large → small type using `(type)`

```
int i = 9;
double d = i;         // widening
int n = (int) d;      // narrowing
//int j = 5.5; //Illegal
int j = (int)5.5;
```

Input with Scanner

- Import and use:

```
import java.util.Scanner;
Scanner input = new Scanner(System.in);
int x = input.nextInt();
input.close();
```

- Methods: `nextInt()`, `nextDouble()`, `nextFloat()`, `nextLine()`, `next()`

Method	Reads	Stops at	Includes spaces?	Example Input `"Hello World"`	Output
`next()`	Next token	Whitespace	No	`"Hello World"`	`"Hello"`
`nextLine()`	Entire line	Newline (`\n`)	Yes	`"Hello World"`	`"Hello World"`

Output Formatting

- `System.out.println()` for simple output
- `System.out.printf()` for formatted output:
  - `%d` for integers
  - `%.3f` for floating-point rounded to 3 decimals

Specifier	Type	Example
`%d`	Integer	`System.out.printf("Age: %d\n", 25);` → `Age: 25`
`%f`	Floating-point	`System.out.printf("Price: %f\n", 12.3456);` → `Price: 12.345600`
`%.3f`	Floating-point rounded to 3 decimals	`System.out.printf("Price: %.3f\n", 12.3456);` → `Price: 12.346`
`%s`	String	`System.out.printf("Name: %s\n", "Alice");` → `Name: Alice`